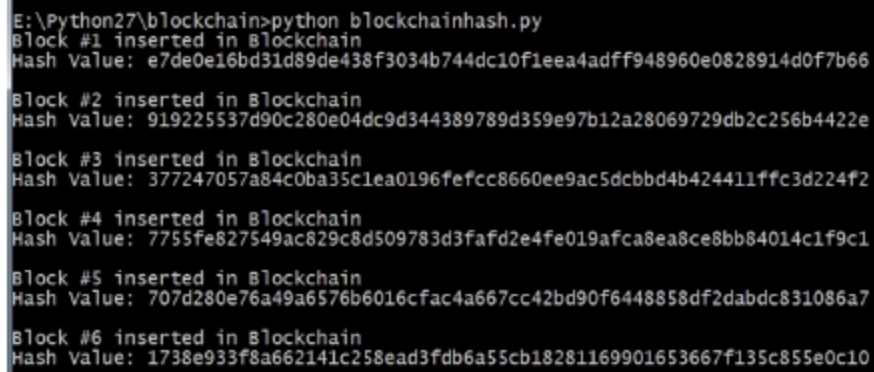


```

back_block = blockchain[0]
maxblocks = 20
for i in range(0, maxblocks):
    block_to_add = next_block(back_block)
    blockchain.append(block_to_add)
    back_block = block_to_add
    print "Block #{i} inserted in Blockchain".format(block_to_
add.idx)
    print "Hash Value: {h}\n".format(block_to_add.hash)

```



```

E:\Python27\blockchain>python blockchainhash.py
Block #1 inserted in Blockchain
Hash Value: e7de0e16bd31d89de438f3034b744dc10f1eea4adff948960e0828914d0f7b66
Block #2 inserted in Blockchain
Hash Value: 919225537d90c280e04dc9d344389789d359e97b12a28069729db2c256b4422e
Block #3 inserted in Blockchain
Hash Value: 377247057a84c0ba35c1ea0196fefcc8660ee9ac5dcbbd4b424411ffc3d224f2
Block #4 inserted in Blockchain
Hash Value: 7755fe827549ac829c8d509783d3fafd2e4fe019afca8ea8ce8bb84014c1f9c1
Block #5 inserted in Blockchain
Hash Value: 707d280e76a49a6576b6016cfac4a667cc42bd90f6448858df2dabdc831086a7
Block #6 inserted in Blockchain
Hash Value: 1738e933f8a662141c258ead3fdb6a55cb18281169901653667f135c855e0c10

```

Figure 2: Generation of hash values using blockchain implementation

With the execution of the code, the outcome shown in Figure 2 is obtained with different hash values, and it provides a higher degree of security using cryptography functions. Using these hash values, hacking or sniffing the transaction is almost impossible.

Deployment of network based distributed blockchains

As in the earlier example, the implementation of the hash function with the blocks is done on a standalone system. In case of the actual blockchain, it is required to be distributed so that different users can initiate their transactions and blocks.

For distributed and Web based implementations, there are different frameworks in Python. Flask is one of the prominent and popular Web frameworks used with Python programming.

```
$ pip install flask
```

In the Windows OS, the following instruction is used to install Flask with an existing Python installation:

```
WindowsDrive:\Python Installation Directory>python -m pip
install flask
```

In blockchain programming, the Proof-of-Work (PoW) is one of the very important algorithms. It is used to confirm and validate the transactions so that the new blocks are added in the blockchain. It is referred to as the key consensus algorithm for the verification and authenticity of the transactions. In the blockchain network, different miners participate for validation and to complete their transactions. For successful validation, the miners are rewarded with digital cryptocurrencies as their remuneration.

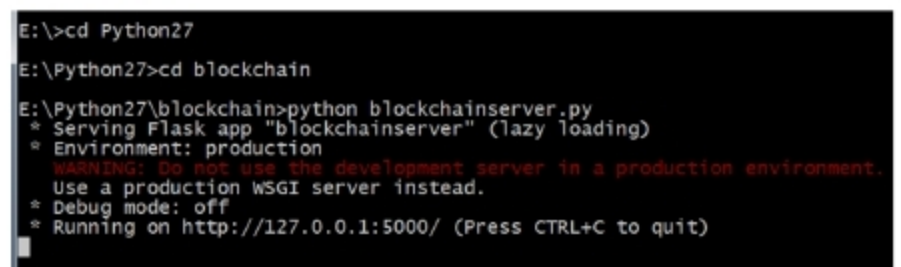
This process also avoids the double spending problem so that the digital currency or transaction is implemented in a secure way. For example, when A transmits a file or digital currency to B, that specific file or currency value in the records of A must be deleted and then should be reflected in the records of B. Traditionally, this is the role performed by the bank as an intermediary. In the case of a blockchain network, it is implemented without any intermediary and is validated automatically using specialised algorithms. If there are instances of the transaction from a sender not getting deleted, it will de-evaluate the currency, whatever is the type of currency used.

```

miner_address = "*****"
myblockchain = []
myblockchain.append(create_genesis_block())
this_nodes_transactions = []
peer_nodes = []
mining = True
@app.route('/myblockchain', methods=['POST'])
def transaction():
    new_myblockchain = request.get_json()
    this_nodes_transactions.append(new_myblockchain)
    print "New transaction"
    print "Sender: {s}".format(new_myblockchain['from'].
encode('ascii','replace'))
    print "Receiver: {r}".format(new_myblockchain['to'].
encode('ascii','replace'))
    print "Amount: {a}\n".format(new_myblockchain['amount'])
    return "Transaction Successful\n"
@app.route('/blocks', methods=['GET'])
def get_blocks():
    chain_to_send = myblockchain
    for i in range(len(chain_to_send)):
        block = chain_to_send[i]
        block_idx = str(block.idx)
        block_timestamp = str(block.timestamp)
        block_data = str(block.data)
        block_hash = block.hash
        chain_to_send[i] = {
            "idx": block_idx,
            "timestamp": block_timestamp,
            "data": block_data,
            "hash": block_hash
        }

```

Figure 3 depicts the Web based implementation of the



```

E:\>cd Python27
E:\Python27>cd blockchain
E:\Python27\blockchain>python blockchainserver.py
* Serving Flask app "blockchainserver" (lazy loading)
* Environment: production
  WARNING: Do not use the development server in a production environment.
  Use a production WSGI server instead.
* Debug mode: off
* Running on http://127.0.0.1:5000/ (Press CTRL+C to quit)

```

Figure 3: Executing the Web based blockchain code

Continued on Page 71....